Actividad: Comunicación en redes

Concepto	Definición Simple	Nivel Técnico (Breve)	Ejemplo Real
TCP	Mensajería con confirmación de entrega.	Protocolo orientado a **conexión** y fiabilidad.	Enviar un **Email** o un archivo (no puede faltar ni una letra).
UDP	Envío rápido sin esperar respuesta.	Protocolo **sin conexión**, prioriza la velocidad.	Una **Videollamada** (si se pierde un frame, la imagen salta y sigue).
Puerto	Las "puertas" numeradas de un edificio.	Punto final lógico para identificar un **proceso** específico.	El **Puerto 80** para web o el **22** para entrar por terminal (SSH).
HTTP	El lenguaje para pedir y enviar webs.	Protocolo de la **capa de aplicación** (Capa 7 OSI).	Escribir **https://www.google.com/search?q=google.com** en el navegador para ver la página.

"Para ver una web, usas el lenguaje **HTTP**, que viaja de forma segura por el camino **TCP**, entrando al servidor por el **Puerto 80**."

⚙️ 2. TCP vs UDP (diferencia clave)

Responder:

- ¿Cuál es la principal diferencia entre TCP y UDP?
- ¿Cuál es más confiable? ¿por qué?
- ¿Cuál es más rápido? ¿por qué?

Mi respuesta

Protocolo | característica clave | uso típico
tcp | protocolo de conexion a la seguridad | mails
udp | protocolo de conexion a la velocidad | stream

Tu tabla va por muy buen camino. Para que tu guía sea 100% precisa y profesional, vamos a ajustar un par de términos técnicos (como cambiar "conexión a la seguridad" por **"orientado a conexión"**), ya que TCP no es seguro por sí mismo (para eso está el SSL/TLS), sino que es **confiable**.

Aquí tienes la versión pulida y expandida para tu documento:

⚡ Comparativa de Protocolos de Transporte

Protocolo	**Característica Clave**	**¿Cómo funciona? (Analogía)**	**Uso Típico**
TCP	**Confiabilidad** (No pierde datos)	Como una **llamada telefónica**: "Hola, ¿me oyes?", "Sí, te oigo".	Mails, Web (HTTP), Transferencia de archivos (FTP).
UDP	**Velocidad** (Sin esperas)	Como un **megáfono**: Tú hablas y el que escuchó, escuchó.	Streaming, Videojuegos online, Videollamadas (Zoom/Meet).

Extras por protocolos:

🔒 SSL y TLS: La Capa de Seguridad

Aunque solemos decir "SSL", hoy en día el estándar que usamos todos es **TLS**. SSL es la versión antigua (ya obsoleta) y TLS es su sucesor moderno y seguro.

¿Para qué sirven? (Los 3 objetivos)

1. **Cifrado (Privacidad):** Se aseguran de que nadie (ni hackers, ni tu proveedor de internet) pueda leer los datos que envías (como contraseñas o tarjetas de crédito).
2. **Autenticación (Identidad):** Confirman que el servidor al que te conectas es realmente quien dice ser (que `google.com` sea realmente Google y no un clon).
3. **Integridad:** Garantizan que los datos no han sido manipulados o alterados durante el viaje por la red.

👉 Ejemplo Developer: HTTPS

Cuando ves una URL que empieza con **HTTPS**, significa que es **HTTP + TLS**.

- Si configuras un servidor (Nginx/Apache), tendrás que instalar un certificado (usualmente de *Let's Encrypt*) para que el puerto **443** funcione con TLS.

"Solo sabia que https es seguro y que http no lo es"

🚪 3. Puertos (clave en desarrollo)

Responder:

- ¿Qué es un puerto en una red?
- ¿Por qué una aplicación necesita un puerto?
- ¿Qué significan estos puertos?
  - 80
  - 443
  - 3000
  - 5432

3. Puertos: La puerta de entrada a tu App

¿Qué es un puerto en una red?

Es un número lógico (del **0 al 65535**) que permite a un sistema operativo entregar los datos al programa correcto. Mientras que la IP identifica la máquina, el puerto identifica el **proceso** o servicio dentro de esa máquina.

¿Por qué una aplicación necesita un puerto?

Sin puertos, el servidor no sabría qué hacer con la información. Si te llega un paquete de datos, el sistema operativo mira el número de puerto para saber si debe enviárselo al Servidor Web, a la Base de Datos o al sistema de Emails. **Es lo que permite la multitarea en red.**

🔑 Diccionario de Puertos Comunes

Puerto	Protocolo / Servicio	Significado para el Dev
80	**HTTP**	Tráfico web estándar (no seguro). Ya casi no se usa solo.
443	**HTTPS**	Tráfico web **seguro (con TLS/SSL)**. Es el estándar actual.
3000	**Desarrollo (Node, React)**	Puerto típico de *localhost* para ver tu app mientras programas.
5432	**PostgreSQL**	El puerto por defecto para conectarte a tu base de datos Postgres.

4. HTTP (HyperText Transfer Protocol)

Responder:

- ¿Qué es HTTP?
- ¿Cómo funciona una petición HTTP?
- ¿Qué significan:
    - GET
    - POST
    - PUT
    - DELETE

🌐 HTTP: El Lenguaje de la Web

¿Qué es HTTP? (HyperText Transfer Protocol)

Es el protocolo estándar que permite la transferencia de datos en la World Wide Web. Es un sistema de **Cliente-Servidor**: tu navegador (cliente) hace una petición y el servidor (donde está la web) responde.

¿Cómo funciona una petición HTTP?

Funciona mediante un ciclo de **Request** (Petición) y **Response** (Respuesta):

1. **Request:** El cliente envía un mensaje al servidor con un **Método** (qué quiere hacer) y una **URL** (a dónde quiere ir).
2. **Procesamiento:** El servidor recibe la petición, busca la información o ejecuta una acción en la base de datos.
3. **Response:** El servidor devuelve un **Código de Estado** (como el famoso 404 o el exitoso 200) y el contenido (HTML, JSON o imágenes).

🛠️ Los 4 Verbos (Métodos) Principales

Como developer, usarás estos cuatro para realizar operaciones **CRUD** (Create, Read, Update, Delete):

Método	Acción Real	Analogía	Ejemplo Developer
GET	**Leer / Obtener**	Consultar un libro en la biblioteca.	Ver el perfil de un usuario o una lista de productos.
POST	**Crear / Enviar**	Rellenar un formulario de inscripción y entregarlo.	Registrar un nuevo usuario o enviar un comentario.

PUT	**Actualizar**	Cambiar un mueble viejo por uno nuevo en tu casa.	Editar tu nombre de usuario o cambiar tu foto de perfil.
DELETE	**Eliminar**	Tirar algo a la basura.	Borrar un post o desactivar una cue

💻 5. Conexión directa con desarrollo (CRÍTICO)

Responder:

- ¿Qué pasa cuando haces:

fetch("<http://localhost:3000/api>")

- ¿Qué protocolo se usa?
- ¿Qué puerto?
- ¿Qué tipo de comunicación?

💻 5. Conexión Directa: El viaje de un `fetch`

Cuando ejecutas `fetch("http://localhost:3000/api")` en tu consola o código, ocurre lo siguiente:

1. ¿Qué protocolo se usa?

Se usan varios en diferentes capas (siguiendo el modelo OSI):

- **Capa de Aplicación: HTTP** (específicamente una petición tipo **GET** por defecto). Es el lenguaje en el que pides los datos.
- **Capa de Transporte: TCP**. El navegador abre un "socket" o conexión bidireccional segura para asegurar que el JSON de la API llegue completo y en orden.

2. ¿Qué puerto?

- El puerto es el **3000**.
- **Dato clave:** Como la URL dice `http` (y no `https`), el navegador asume que no hay cifrado TLS inicial, pero busca específicamente el proceso que "escucha" en el anexo 3000 de tu propia máquina.

3. ¿Qué tipo de comunicación?

Es una comunicación **Cliente-Servidor** de tipo **Modelo de Petición-Respuesta (Request-Response)**.

- **Localhost:** Tu computadora actúa como cliente (el navegador) y como servidor (tu proceso de Node/Python/Go) al mismo tiempo.
- **Sincrónica/Asincrónica:** Desde el punto de vista del código, es **asincrónica** (por eso usas `await` o `.then`), ya que el procesador no se bloquea esperando a que la red responda.

🚨 6. Problemas reales (debugging)

Responder:

- ¿Qué significa:
  - “Port already in use”?
  - “Connection refused”?
  - “Timeout”?
- ¿Qué revisarías como developer?

🛠️ Guía de Debugging: Los 3 Errores Fatales

1. "Port already in use" (EADDRINUSE)

- **Qué significa:** Intentaste levantar tu app (ej. en el puerto `3000`), pero **otra aplicación** ya "secuestró" ese puerto. Dos procesos no pueden escuchar en la misma "puerta" al mismo tiempo.
- **Causa común:** Dejaste una pestaña de la terminal abierta con el servidor corriendo, o un proceso se quedó colgado en segundo plano.
- **¿Qué revisar?**
  - Cerrar otras terminales.
  - Usar un comando para matar el proceso (ej. `lsof -i :3000` y luego `kill`).
  - Cambiar el puerto en tu archivo `.env`.

2. "Connection refused" (ECONNREFUSED)

- **Qué significa:** Llegaste a la dirección IP correcta, pero **nadie estaba escuchando** en ese puerto. El servidor rechazó la conexión inmediatamente.
- **Causa común:** Se te olvidó ejecutar `npm start` (el servidor está apagado) o te equivocaste de puerto (la app está en el `3000` y tú pides el `8080`).

- **¿Qué revisar?**
    - ¿Está el proceso del servidor encendido?
    - ¿El puerto en la URL coincide con el puerto donde corre la app?
    - ¿El firewall está bloqueando ese puerto específico?

3. "Timeout" (ETIMEDOUT)

- **Qué significa:** Tu petición salió, pero **nunca recibió respuesta**. El cliente esperó (usualmente 30 segundos) y se rindió. Es como llamar por teléfono y que nadie atienda ni dé señal de ocupado.
- **Causa común:** Un **Firewall** está "dropeando" (ignorando) tus paquetes, o el servidor está tan saturado que no puede ni responder.
- **¿Qué revisar?**
    - **Conectividad:** ¿Tienes internet? ¿La VPN está activa?
    - **Reglas de Red:** En AWS/Azure, revisa los *Security Groups*.
    - **Latencia:** ¿El servidor está muy lejos o procesando algo muy pesado?

🌍 7. Caso práctico real

Escenario:

> “Tu frontend no se conecta al backend”

Responder:

- ¿Puede ser problema de puerto?
- ¿Puede ser problema de protocolo?
- ¿Puede ser problema de red?

🌍 7. Caso Práctico: "Frontend vs Backend"

1. ¿Puede ser problema de puerto? (Altamente Probable)

- **La razón:** El frontend suele correr en un puerto (ej. `3000`) y el backend en otro (ej. `8080` o `5000`).
- **El error:** Si en tu código de frontend haces `fetch("http://localhost:5000")` pero tu backend en realidad inició en el `5001`, la conexión fallará inmediatamente con un **Connection Refused**.
- **Qué revisar:** Los logs de la terminal del backend para confirmar en qué puerto está "escuchando".

2. ¿Puede ser problema de protocolo? (Muy Probable)

- **La razón:** Existe un guardián llamado **CORS** (Cross-Origin Resource Sharing) y las reglas de **HTTPS**.

- **El error:** * Si tu frontend está en una web segura (`https://`) e intenta llamar a un backend no seguro (`http://`), el navegador bloqueará la petición por "Mixed Content".
    - También puede ser un error de **CORS**: el navegador bloquea la respuesta porque el backend no ha dado permiso explícito al dominio del frontend para leer sus datos.
- **Qué revisar:** La pestaña "Console" y "Network" de las herramientas de desarrollador (F12).

3. ¿Puede ser problema de red? (Probable en Despliegue)

- **La razón:** En local (localhost) la red casi nunca falla, pero en la nube todo cambia.
- **El error:** * **IPs Privadas:** Intentar conectar el frontend (que corre en el navegador del cliente) a una IP privada del servidor (ej. `10.0.0.5`) que no es visible desde internet.
    - **Firewalls/Security Groups:** El servidor del backend tiene el puerto cerrado para conexiones externas.
- **Qué revisar:** ¿Puedes llegar al backend usando herramientas como **Postman** o **Thunder Client**? Si ellos tampoco llegan, es un bloqueo de red o firewall.

ANALOGÍA

Concepto	Tu Analogía	Definición Técnica	Por qué te importa como Dev
TCP	**Llamada telefónica**	Protocolo orientado a conexión.	Si falta un bit de tu JSON, la conexión lo reintenta hasta que llegue perfecto.
UDP	**Mensaje sin confirmación**	Protocolo sin conexión (rápido).	Ideal para juegos o video donde prefieres perder un frame antes que congelar la imagen.
Puerto	**Número de departamento**	Identificador de proceso en el SO.	Es donde "escucha" tu servidor (Node en 3000, Python en 8000, Postgres en 5432).
HTTP	**Idioma de comunicación**	Protocolo de capa de aplicación.	Es el estándar para enviar y recibir datos (GET, POST, PUT, DELETE) en la web.